

memtorch.bh.nonideality

Submodule containing various models, which can be used to introduce various non-ideal device characteristics using `memtorch.bh.nonideality.NonIdeality.apply_nonidealities`.

memtorch.bh.nonideality.NonIdeality

Class used to introduce/model non-ideal device and circuit characteristics. `patched_model.apply_nonidealities` is commonly used to introduce such characteristics, as demonstrated by the following example:

```
import copy
import Net
from memtorch.nn.Module import patch_model
from memtorch.map.Parameter import naive_map
from memtorch.map.Inout import naive_scale

model = Net()
reference_memristor = memtorch.bh.memristor.VTEAM
patched_model = patch_model(copy.deepcopy(model),
                             memristor_model=reference_memristor,
                             memristor_model_params={},
                             module_parameters_to_patch=[torch.nn.Linear, torch.nn.Conv2d],
                             mapping_routine=naive_map,
                             transistor=True,
                             programming_routine=None,
                             tile_shape=(128, 128),
                             max_input_voltage=8.3,
                             scaling_routine=naive_scale,
                             ADC_resolution=8,
                             ADC_overflow_rate=0.,
                             quant_method='linear')
# Example usage of memtorch.bh.nonideality.NonIdeality.DeviceFaults
patched_model = patched_model.apply_nonidealities(patched_model,
                                                  non_idealities=[memtorch.bh.nonideality.NonIdeality.DeviceFaults],
                                                  lrs_proportion=0.25,
                                                  hrs_proportion=0.10,
                                                  electroform_proportion=0)
```

class memtorch.bh.nonideality.NonIdeality.NonIdeality [\[source\]](#)

Bases: `enum.Enum`

Nonideality enumeration.

DeviceFaults= 2

Endurance= 4

FiniteConductanceStates= 1

NonLinear= 3

Retention= 5

memtorch.bh.nonideality.NonIdeality.apply_nonidealities(model, non_idealities, **kwargs) [\[source\]](#)

Method to apply non-idealities to a `torch.nn.Module` instance with memristive layers.

- Parameters:
- `model` (`torch.nn.Module`) – `torch.nn.Module` instance.
 - `nonidealities` (`memtorch.bh.nonideality.NonIdeality.NonIdeality, tuple`) – Non-linearities to model.

Returns: Patched instance.

Return type: `torch.nn.Module`

memtorch.bh.nonideality.NonIdeality.required(kwargs, arguments, call) [\[source\]](#)

Method to check if required arguments in `**kwargs` are present.

- Parameters:
- `kwargs` (`**kwargs`) – Keyword-arguments.
 - `arguments` (`list of str`) – Arguments which are required to be present.
 - `call` (`str`) – Function to call.

memtorch.bh.nonideality.FiniteConductanceStates

memtorch.bh.nonideality.FiniteConductanceStates.apply_finite_conductance_states(layer, n_conductance_states) [\[source\]](#)

Method to model a finite number of conductance states for devices within a memristive layer.

- Parameters:
- `layer` (`memtorch.mn`) – A memrstive layer.
 - `n_conductance_states` (`int`) – Number of finite conductance states to model.

Returns: The patched memristive layer.

Return type: `memtorch.mn`

memtorch.bh.nonideality.DeviceFaults

Methods used to model device faults.

memtorch.bh.nonideality.DeviceFaults.apply_cycle_variability(layer, distribution=<class 'torch.distributions.normal.Normal'>, min=0, max=inf, parallelize=False, r_off_kwargs={}, r_on_kwargs={}) [\[source\]](#)

Method to apply cycle-to-cycle variability to a memristive layer.

- Parameters:
- `layer` (`memtorch.mn`) – A memrstive layer.
 - `distribution` (`torch.distributions`) – `torch` distribution.
 - `min` (`float`) – Minimum value to sample.
 - `max` (`float`) – Maximum value to sample.
 - `parallelize` (`bool`) – The operation is parallelized (True).
 - `r_off_kwargs` (`dict`) – `r_off` kwargs.
 - `r_on_kwargs` (`dict`) – `r_on` kwargs.

memtorch.bh.nonideality.DeviceFaults.apply_device_faults(layer, lrs_proportion, hrs_proportion, electroform_proportion) [\[source\]](#)

Method to model device failure within a memristive layer.

- Parameters:
- `layer` (`memtorch.mn`) – A memrstive layer.
 - `lrs_proportion` (`float`) – Proportion of devices which become stuck at a low resistance state.
 - `hrs_proportion` (`float`) – Proportion of devices which become stuck at a high resistance state.
 - `electroform_proportion` (`float`) – Proportion of devices which fail to electroform.

Returns: The patched memristive layer.

Return type: `memtorch.mn`

memtorch.bh.nonideality.NonLinear

memtorch.bh.nonideality.NonLinear.apply_non_linear(layer, sweep_duration=1, sweep_voltage_signal_amplitude=1, sweep_voltage_signal_frequency=1, num_conductance_states=None, simulate=False) [\[source\]](#)

Method to model non_linear I/V characteristics for devices within a memristive layer.

- Parameters:
- `layer` (`memtorch.mn`) – A memrstive layer.
 - `sweep_duration` (`float`) – Voltage sweep duration (s).
 - `sweep_voltage_signal_amplitude` (`float`) – Voltage sweep amplitude (V).
 - `sweep_voltage_signal_frequency` (`float`) – Voltage sweep frequency (Hz).
 - `num_conductance_states` (`int, optional`) – Number of finite conductance states to model. None indicates finite states are not to be modeled.
 - `simulate` (`bool, optional`) – Each device is simulated during inference (True).
- Returns: The patched memristive layer.
- Return type: `memtorch.mn`

memtorch.bh.nonideality.Endurance

memtorch.bh.nonideality.Endurance.apply_endurance_model(layer, x, endurance_model=<function model_endurance_retention>, **endurance_model_kwargs) [\[source\]](#)

Method to apply an endurance model to devices within a memristive layer.

- Parameters:
- `layer` (`memtorch.mn`) – A memrstive layer.
 - `x` (`float`) – Energy (J) / SET-RESET cycles.
 - `endurance_model` (`function`) – Endurance model to use.
 - `endurance_model_kwargs` (`**kwargs`) – Endurance model keyword arguments.
- Returns: The patched memristive layer.
- Return type: `memtorch.mn`

memtorch.bh.nonideality.Retention

For both `memtorch.bh.nonideality.Endurance` and `memtorch.bh.nonideality.Retention`, the following internal endurance and retention models are natively supported:

memtorch.bh.nonideality.endurance_retention_models.conductance_drift

Conductance drift model as described in: I. Boybat et al., "Impact of conductance drift on multi-PCM synaptic architectures", 2018 Non-Volatile Memory Technology Symposium (NVMTS), 2018.

memtorch.bh.nonideality.endurance_retention_models.conductance_drift.model_conductance_drift(layer, x, initial_time=1e-12, drift_coefficient=0.1) [\[source\]](#)

- Parameters:
- `layer` (`memtorch.mn`) – A memrstive layer.
 - `x` (`float`) – Retention time (s). Denoted using `x` for sake of consistency with other models.
 - `initial_time` (`float`) – Initial time that corresponds with initial conductance values.
 - `drift_coefficient` (`float`) – Drift coefficient. For PCM devices, `drift_coefficient` typically has a value of 0.1 for the amorphous phase.
- Returns: The patched memristive layer.
- Return type: `memtorch.mn`

memtorch.bh.nonideality.endurance_retention_models.empirical_metal_oxide_RRAM

Empirical Metal-Oxide RRAM Device Endurance and Retention Model as described in: C. Lammie, M. Rahimi Azghadi and D. Ielmini, "Empirical Metal-Oxide RRAM Device Endurance and Retention Model For Deep Learning Simulations", Semiconductor Science and Technology, 2021.

class memtorch.bh.nonideality.endurance_retention_models.empirical_metal_oxide_RRAM.OperationMode [\[source\]](#)

Bases: `enum.Enum`

An enumeration.

gradual= 2

sudden= 1

memtorch.bh.nonideality.endurance_retention_models.empirical_metal_oxide_RRAM.model_endurance_retention(layer, operation_mode, x, p_lrs, stable_resistance_lrs, p_hrs, stable_resistance_hrs, cell_size, temperature, temperature_threshold=298) [\[source\]](#)

Method to model endurance and retention characteristics.

- Parameters:
- `layer` (`memtorch.mn`) – A memrstive layer.
 - `operation_mode` (`memtorch.bh.endurance_retention_models.OperationMode`) – Failure operational mode (sudden or gradual).
 - `x` (`float`) – `x` parameter.
 - `p_lrs` (`list`) – Low resistance state `p_0`, `p_1`, `p_2`, and `p_3` values.
 - `stable_resistance_lrs` (`float`) – Stable low resistance state.
 - `p_hrs` (`list`) – High resistance state `p_0`, `p_1`, `p_2`, and `p_3` values.
 - `stable_resistance_hrs` (`float`) – Stable high resistance state.
 - `cell_size` (`float`) – Device cell size (nm).
 - `temperature` (`float`) – Operational temperature (K).
 - `temperature_threshold` (`float`) – Temperature threshold (K) in which the device begins to fail.
- Returns: The patched memristive layer.
- Return type: `memtorch.mn`

memtorch.bh.nonideality.endurance_retention_models.empirical_metal_oxide_RRAM.model_endurance_retention_gradual(initial_resistance, x, p_0, p_1, p_2, p_3, threshold, temperature_constant, cell_size=None) [\[source\]](#)

Method to model gradual endurance_retention failure.

- Parameters:
- `initial_resistance` (`tensor`) – Initial resistance values. `x` : float
 - Energy (J) / SET-RESET cycles / Retention time (s).
 - `p_0` (`float`) – `p_0` fitting parameter.
 - `p_1` (`float`) – `p_1` fitting parameter.
 - `p_2` (`float`) – `p_2` fitting parameter.
 - `p_3` (`float`) – `p_3` fitting parameter.
 - `threshold` (`float`) – Threshold for `x`.
 - `temperature_threshold` (`float`) – Temperature threshold (K) in which the device begins to fail.
 - `cell_size` (`float`) – Device cell size (nm).
- Returns: Updated resistance values.
- Return type: `tensor`

memtorch.bh.nonideality.endurance_retention_models.empirical_metal_oxide_RRAM.scale_p_0(p_0, p_1, v_stop, v_stop_min, v_stop_max, v_stop_optimal, cell_size=10) [\[source\]](#)

Method to scale `p_0` to introduce dependence on `v_stop`

- Parameters:
- `p_0` (`float`) – Unscaled `p_0` parameter.
 - `p_1` (`float`) – `p_1` parameter.
 - `v_stop` (`float`) – `v_stop` value to determine `p_0` for.
 - `v_stop_min` (`float`) – minimum `v_stop` value.
 - `v_stop_max` (`float`) – maximum `v_stop` value.
 - `v_stop_optimal` (`float`) – optimal `v_stop` value (endurance).
 - `cell_size` (`float`) – Device cell size (nm).
- Returns: Scaled `p_0` value.
- Return type: `float`